

RMIT

ISYS3476 (Postgraduate)
Managing Semi-structured and Unstructured Data
Assignment 1 Part B: Task 1 Semantics and
Examples

Purpose. This optional reference revisits the Task 1 semantics taught in class through worked examples and edge-case illustrations. Use it when you would like more detail or want to check your understanding. Task 2 and Task 3 requirements are stated in the Main Specification and rubric.

Implementation note. The examples illustrate expected behaviour and output formats. The stored package must remain compatible with the provided builder and access helpers.

Contents

1	Task 1 Example Corpus	2
2	Index Construction and Access	2
2.1	Package schema	2
2.2	Character n -grams with \$ boundaries	3
2.3	Access-function edge cases	3
2.4	Toy corpus checks	3
3	Query Processing Semantics	3
3.1	Boolean parsing notes	3
3.2	Wildcard semantics	4
3.3	NEAR/ k semantics	4
4	Query Examples	4
4.1	Boolean query examples	4
4.2	Wildcard query examples	4
4.3	Proximity query examples	4

1 Task 1 Example Corpus

To reduce cognitive load, most examples in this appendix use the same tiny corpus as the Main Spec:

```
doc_ids = [191, 279]
tokenized_docs = [
  ["climate", "change", "policy"],      # doc 191
  ["machine", "learning", "improves"]   # doc 279
]
```

2 Index Construction and Access

2.1 Package schema

The index package must remain compatible with the provided builder and access helpers. The following example shows the required top-level keys and value shapes. You may add fields without changing these contracts.

```
{
  "__META__": {
    "N": 2,
    "doc_ids": [191, 279],
    "doc_lengths": {191: 3, 279: 3},
    "avgdl": 3.0,
    "version": "1.0",
    "char_ngrams_max": 3
  },

  "unified": {
    "climate": {"df": 1, "postings": {191: {"tf": 1}}},
    "change": {"df": 1, "postings": {191: {"tf": 1}}},
    "policy": {"df": 1, "postings": {191: {"tf": 1}}},
    "machine": {"df": 1, "postings": {279: {"tf": 1}}},
    "learning": {"df": 1, "postings": {279: {"tf": 1}}},
    "improves": {"df": 1, "postings": {279: {"tf": 1}}}
  },

  "wildcard": {
    "$cl": ["climate"],
    "ng$": ["learning"],
    ...
  },

  "proximity": {
    "climate": {191: [0]},
    "learning": {279: [1]},
    ...
  }
}
```

The unified term index is provided. Its document-frequency and term-frequency values support optional ranking methods in Task 2 and are not an additional Task 1 requirement.

2.2 Character n -grams with \$ boundaries

For each vocabulary term (e.g., "climate"), form a bounded string:

$$s = \$ + \text{term} + \$.$$

Index all character n -grams of length 1–3 from s . **Exclude indexing the unigram "\$".**

Why exclude "\$"? If "\$" were indexed alone, it would match every term (every bounded string contains \$), which makes it semantically meaningless and harms wildcard expansion.

2.3 Access-function edge cases

- Missing key: return [].
- Output ordering requirements:
 - `get_posting_list`: strictly ascending numeric doc IDs.
 - `find_wildcard_matches`: lexicographically ascending terms.
 - `get_term_positions`: strictly ascending integer positions.
- Determinism: repeated runs on the same input must yield the same on-disk package and the same outputs.

2.4 Toy corpus checks

The following access and query examples use the index produced from the toy corpus above and saved as `index_pkg.pkl`.

```
# Unigram postings
get_posting_list("climate", "index_pkg.pkl")    # -> [191]
get_posting_list("learning", "index_pkg.pkl")    # -> [279]

# Wildcard term expansion
find_wildcard_matches("$cl", "index_pkg.pkl")    # -> ["climate"]
find_wildcard_matches("ng$", "index_pkg.pkl")    # -> ["learning"]

# Positions
get_term_positions("change", 191, "index_pkg.pkl") # -> [1]
get_term_positions("learning", 279, "index_pkg.pkl") # -> [1]

# OOV and missing-pair lookups return empty lists
get_posting_list("algorithms", "index_pkg.pkl")    # -> []
get_term_positions("climate", 279, "index_pkg.pkl") # -> []
```

3 Query Processing Semantics

3.1 Boolean parsing notes

- Precedence: NOT > AND > OR.
- Parentheses override precedence.
- Recommended approach: convert to postfix (shunting-yard) then evaluate using set operations.

3.2 Wildcard semantics

A wildcard pattern contains exactly one `*`, which matches any sequence of characters, including the empty sequence. Use the wildcard sub-index to obtain candidate terms, then return the union of postings for candidates matching the complete pattern. The choice of candidate n -grams is part of your implementation.

3.3 NEAR/ k semantics

Treat each operand as a unigram occurrence at position p and q . A document satisfies NEAR/ k iff there exist occurrences such that:

$$D = |p - q| \leq k.$$

Adjacency is $D = 1$. For unigram operands, two distinct occurrences cannot share one position, so NEAR/0 returns no matches. If both operands are the same term, p and q must refer to two distinct occurrences. This definition is order-insensitive.

4 Query Examples

4.1 Boolean query examples

```
process_boolean_query("climate AND change", "index_pkg.pk1")
# -> {191}

process_boolean_query("( climate OR machine ) AND NOT policy", "index_pkg.
pk1")
# climate OR machine -> {191, 279}; NOT policy removes doc 191 -> {279}

process_boolean_query("machine AND algorithms", "index_pkg.pk1")
# -> set() (the token 'algorithms' is OOV)
```

4.2 Wildcard query examples

```
process_wildcard_query("climat*", "index_pkg.pk1") # prefix
# -> {191}

process_wildcard_query("*ing", "index_pkg.pk1") # suffix
# -> {279}

process_wildcard_query("learn*ing", "index_pkg.pk1") # internal
# -> {279}
```

4.3 Proximity query examples

```
process_proximity_query("climate NEAR/1 change", "index_pkg.pk1")
# -> {191}
process_proximity_query("climate NEAR/3 policy", "index_pkg.pk1")
# -> {191}
process_proximity_query("climate NEAR/0 change", "index_pkg.pk1")
# -> set() (the occurrence distance is 1, so k=0 fails)
```